MODULE 21 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

SQL features, semi-structured and unstructured data

Three kinds of data, one SQL dialect. QUALIFY, MERGE, VARIANT, FLATTEN and file URLs.

DOMAIN 4 of 5 WEIGHT 21% MODULE 21 of 31

admin@sthithapragna.com

QUALIFY

- **QUALIFY filters on the result of a window function**, the way HAVING filters on the result of an aggregate.
- You cannot put a window function in a WHERE clause. Without QUALIFY you need a subquery or a CTE.
- The classic use: **keep only the latest row per key**, with ROW_NUMBER partitioned by the key and ordered by timestamp.
- Evaluated **after** window functions are computed, which is why it can see them.

> ON THE EXAM
>
> If a scenario describes deduplicating to the most recent record and offers QUALIFY, that is almost always the intended answer.

Reshaping and merging

- **PIVOT** turns rows into columns; **UNPIVOT** turns columns into rows. Both need the value set specified, or ANY.
- **MERGE** combines insert, update and delete against a target using a source and an ON condition. The upsert.
- MERGE is what a **stream plus task** pipeline uses to apply captured changes, from module 15.
- **Multi-table INSERT** writes one source into several targets in a single statement, conditionally or unconditionally.

> ON THE EXAM
>
> MERGE is the reason streams and tasks beat dynamic tables when the logic is complex. Dynamic tables do not support MERGE.

Sampling and a few others

- **SAMPLE** or **TABLESAMPLE** returns a subset. By fraction of rows, by a fixed number of rows, or by block.
- Row sampling is more accurate; **block sampling is faster** because it selects whole micro-partitions.
- **Common table expressions** with WITH, including **recursive** CTEs for hierarchies.
- **Window functions**: ROW_NUMBER, RANK, LAG, LEAD, and running totals with OVER.

> ON THE EXAM
>
> Sampling is how you test a transformation against a large table without paying to scan all of it.

VARIANT, OBJECT and ARRAY

- **VARIANT** holds any semi-structured value. **OBJECT** is a set of key-value pairs. **ARRAY** is an ordered list.
- **VARIANT is limited to 16 MB per row**, compressed. That bounds how large one document can be.
- **PARSE_JSON** converts a string to a VARIANT. **TO_VARIANT** converts any value. **TO_JSON** goes back to a string.
- In a VARIANT, a JSON null is stored as the string 'null', not a SQL NULL. **STRIP_NULL_VALUES** handles that on load.

> ON THE EXAM
>
> The 16 MB limit is a real constraint, not trivia. A scenario about very large JSON documents is usually pointing at it.

Getting at the values

- **Colon** for the first level: **src:user**. **Dot** or **brackets** below that: **src:user.email** or **src['user']['email']**.
- **Always cast.** src:user.email returns a VARIANT; src:user.email::STRING returns a string.
- Without the cast, values come back **with quotes** and comparisons behave oddly.
- Paths are **case-sensitive**. Bracket notation is required when a key contains a space or a special character.

> ON THE EXAM
>
> 'Why does my value have quotes around it' is the commonest semi-structured complaint, and the answer is a missing cast.

FLATTEN parameters

PARAMETER	MEANING AND DEFAULT
INPUT	Required. The expression to flatten. Must be VARIANT, OBJECT or ARRAY.
PATH	The element to flatten. An empty string flattens the outermost element.
OUTER	Default FALSE. Rows that cannot be expanded are omitted entirely. TRUE keeps one row with NULLs.
RECURSIVE	Default FALSE. Only the element at PATH is expanded. TRUE expands all sub-elements.
MODE	Default BOTH. Or restrict to 'OBJECT' or 'ARRAY'.

ON THE EXAM

OUTER is effectively an outer join. With the default FALSE, a row whose array is empty disappears from your results entirely.

FLATTEN output columns

COLUMN	WHAT IT HOLDS
SEQ	A sequence number per input record. Not guaranteed gap-free or ordered.
KEY	The key, for maps and objects. NULL for arrays.
PATH	The path to the element within the structure.
INDEX	The array index, or NULL if it is not an array.
VALUE	The value of the flattened element. This is the one you usually select.
THIS	The element being flattened. Useful in recursive flattening.

ON THE EXAM

Six fixed columns, always the same. Columns of the correlated source table remain accessible alongside them.

LATERAL FLATTEN

- **LATERAL** lets a table function see columns from the row preceding it in the FROM clause.
- Without LATERAL, FLATTEN has no way to know *which* row's JSON to expand.
- **For single-level arrays, TABLE(FLATTEN(...)) and LATERAL FLATTEN(...) give the same result.**
- The usual shape: **FROM raw, LATERAL FLATTEN(input => raw.payload:events)**.

> ON THE EXAM
>
> Think of LATERAL as 'for each row on the left, run this function using that row's data'. It is a correlated join.

Schema detection and evolution

- **INFER_SCHEMA** inspects staged files and returns the column definitions it detects.
- **CREATE TABLE ... USING TEMPLATE** builds a table from that inferred schema, so you need not write the DDL.
- **ENABLE_SCHEMA_EVOLUTION** on a table lets loads add new columns, and drop NOT NULL from columns missing in new files.
- Setting that parameter requires **OWNERSHIP** on the table. It is what the Kafka connector needed in module 16.

> ON THE EXAM
>
> These are the tools for data whose shape you do not control. Standard Edition, and they work with Parquet, Avro, ORC, JSON and CSV.

The three file URL types

TYPE	FUNCTION	EXPIRY	WHO CAN USE IT
File URL	BUILD_STAGE_FILE_URL	Never expires	Authenticated, with read privileges on the stage
Scoped URL	BUILD_SCOPED_FILE_URL	24 hours	Authenticated, but no stage privileges needed
Pre-signed URL	GET_PRESIGNED_URL	Configurable. Default 3600s, max 7 days	Anyone. The token is in the URL

ON THE EXAM

Scoped URLs are created on demand when a consumer queries a secure view. That is how unstructured data is shared.

The encryption trap, and the FILE type

- Files on an internal stage are **client-side encrypted by default**, with keys owned by Snowflake.
- **Client-side encrypted files are unreadable** through pre-signed, file or scoped URLs. They simply will not open.
- Create the stage with **ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')** for server-side encryption. GET_PRESIGNED_URL requires it.
- The **FILE data type** holds a reference to a staged file, carrying STAGE, RELATIVE_PATH, both URL forms, CONTENT_TYPE, SIZE and more.

> ON THE EXAM
>
> This has to be decided when the stage is created. Discovering it after loading a million documents is an expensive morning.

How this module gets tested

THE REQUIREMENT	THE ANSWER
Keep only the latest row per key	QUALIFY with ROW_NUMBER
Apply captured changes: insert, update and delete	MERGE
Turn a JSON array into one row per element	LATERAL FLATTEN
Rows with an empty array are disappearing	OUTER => TRUE on FLATTEN
Values come back with quotes around them	Missing ::cast on the path expression
A partner with no Snowflake account must open a file	Pre-signed URL, and server-side encryption on the stage

ON THE EXAM

Six shapes. The last one needs two facts together, which is why it makes a good question.

> A table holds one row per customer update, with many rows per customer. An analyst needs a result containing only the most recent row for each customer, judged by an updated_at timestamp.
>
> They have written a query using ROW_NUMBER partitioned by customer_id and ordered by updated_at descending, and are filtering with WHERE rn = 1. The statement fails.

Why does it fail, and what is the cleanest fix?

A ROW_NUMBER cannot be partitioned. Use RANK instead and filter on that.

B The ORDER BY must be ascending inside a window function. Reverse it and take the last row.

C Deduplication requires a MERGE into a separate table. A SELECT cannot express it.

D A window function cannot be used in WHERE. Use QUALIFY, which filters on window function results.

◆ PAUSE AND DECIDE

Why does it fail, and what is the cleanest fix?

A ROW_NUMBER cannot be partitioned.

Partitioning is its normal use. The problem is where the filter sits, not the function.

B ORDER BY must be ascending.

Invented. Descending is standard for taking the most recent row.

C Deduplication needs a MERGE.

A SELECT expresses this fine. MERGE writes changes to a target; nothing here is being written.

D Window functions cannot be used in WHERE. Use QUALIFY.

WHERE is evaluated before window functions are computed. QUALIFY runs after, which is exactly why it exists.

THE SIGNAL

A subquery or CTE also works and is more verbose. 'Cleanest fix' is what points at QUALIFY.

A raw table has a VARIANT column named payload, containing an array at payload:items. An engineer writes a query selecting from the raw table and from FLATTEN(input => payload:items), expecting one row per item.

The statement fails with an error about an invalid identifier. Colleagues report that a nearly identical query in another schema works.

What is missing?

A PARSE_JSON must wrap the column, because FLATTEN cannot read a VARIANT directly.

B LATERAL. Without it, FLATTEN cannot see the columns of the row preceding it in the FROM clause.

C The MODE parameter must be set to 'ARRAY' before an array can be flattened.

D The array must first be cast with ::ARRAY, since FLATTEN only accepts the ARRAY type.

◆ PAUSE AND DECIDE

What is missing?

A PARSE_JSON must wrap the column.

PARSE_JSON converts a string to a VARIANT. The column is already a VARIANT, so there is nothing to parse.

B LATERAL, so FLATTEN can see the preceding row.

A table function has no connection to preceding rows without LATERAL. Hence the invalid identifier on payload.

C MODE must be 'ARRAY'.

MODE defaults to BOTH, which already covers arrays. It restricts rather than enables.

D Cast with ::ARRAY first.

FLATTEN accepts VARIANT, OBJECT or ARRAY. The input type is not the problem.

THE SIGNAL

The error names an identifier, not a type. That is the clue: FLATTEN cannot resolve 'payload' because it cannot see that row.

A team stores scanned contracts on a named internal stage created with default settings. They generate pre-signed URLs with GET_PRESIGNED_URL and send one to an external auditor who has no Snowflake account.

The URL resolves, but the downloaded file is unreadable. Opening it in Snowsight from the stage works perfectly.

What is happening, and what is required?

A Internal stage files are client-side encrypted by default. The stage needs server-side encryption, SNOWFLAKE_SSE.

B Pre-signed URLs only work on external stages. Move the contracts to S3.

C The auditor needs a Snowflake account. Pre-signed URLs still require authentication.

D The expiration was too short and the token had lapsed. Increase it to the 7-day maximum.

◆ PAUSE AND DECIDE

What is happening, and what is required?

A Client-side encrypted by default. Needs SNOWFLAKE_SSE.

Documented: client-side encrypted files are unreadable through pre-signed, file or scoped URLs.
GET_PRESIGNED_URL requires server-side encryption.

B Pre-signed URLs need an external stage.

They work on internal stages, provided server-side encryption is configured.

C The auditor needs an account.

The defining property of a pre-signed URL is that no authentication is needed. The token is in the URL.

D The token had lapsed.

An expired token fails to resolve. Here it resolved and delivered a file, which was simply unreadable.

THE SIGNAL

'The URL resolves but the file is unreadable' is the exact fingerprint. An auth failure would not deliver a file at all.

> A pipeline loads device telemetry as JSON into a VARIANT column. Most documents load without issue. A small number of devices produce documents that fail to load, and those devices report very long event histories in a single message.
>
> The team has checked the file format and the JSON validates correctly.

What is the most likely cause?

A Deeply nested JSON requires ENABLE_SCHEMA_EVOLUTION on the target table.

B PARSE_JSON cannot handle arrays with more than 1,000 elements. Use TO_VARIANT instead.

C The documents exceed the VARIANT limit. Split each message into several rows, or store only the events array.

D The COPY statement needs MATCH_BY_COLUMN_NAME, because VARIANT columns match by name.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Nesting requires schema evolution.

Schema evolution adds columns as sources change. It has nothing to do with the size or depth of one document.

B PARSE_JSON caps arrays at 1,000 elements.

Invented. There is no element-count limit, only the overall size limit.

C They exceed the 16 MB VARIANT limit.

VARIANT is capped at 16 MB per row. Long event histories in one message is exactly how that limit is reached.

D MATCH_BY_COLUMN_NAME is needed.

That maps staged columns to target columns by name. It does not affect whether a document fits in a VARIANT.

THE SIGNAL

'Most load fine, a few fail, and those few are the largest' is the shape of a size limit rather than a format problem.

What to carry into the next module

- **QUALIFY filters on window functions**, as HAVING does on aggregates. WHERE cannot see them.
- **LATERAL lets FLATTEN see the current row.** OUTER defaults to FALSE, so unexpandable rows vanish.
- **FLATTEN returns SEQ, KEY, PATH, INDEX, VALUE, THIS.** Always cast path expressions with ::type.
- **VARIANT is 16 MB per row.** A JSON null is stored as the string 'null'.
- **File URL never expires, scoped is 24 hours, pre-signed needs no auth.** Internal stages need SNOWFLAKE_SSE for any of them.

> ON THE EXAM
>
> Next: UDFs, stored procedures and Snowpark, then six scenarios closing Domain 4.